

Graph Foundations — Advanced Module

A comprehensive deep-dive into the theory, implementation, and real-world applications of graph data structures — from formal definitions to production-scale systems.

1

Part 1

Graph Basics

2

Part 2

Matrix as Graph



PART 1: Graph Basics

Section 1: Topic Classification

Field	Details
Topic Name	Graph Basics
Category	Data Structures
Domain / Subdomain	Computer Science / Non-Linear Data Structures / Graph Theory
Topic Type	Conceptual + Structural + Implementation
Difficulty Level	Advanced
Prerequisites	Trees, Recursion, Hash Maps, Arrays, Linked Lists, Asymptotic Analysis
Industry Relevance	Critical — Powers social networks, routing protocols, dependency resolution, fraud detection, recommendation engines, compilers, OS schedulers
Recommended Learning Path	Trees → Graph Basics → BFS → DFS → Shortest Paths → MST → Advanced Graph Algorithms

What Is a Graph?

Section 2: Formal Definition & Key Concepts



A **graph** G is defined as an ordered pair $G = (V, E)$ where V is a finite, non-empty set of **vertices** and E is a set of **edges**, each connecting a pair of vertices.

An edge between vertices u and v is written as (u, v) for directed graphs or $\{u, v\}$ for undirected graphs.

Why Graphs Generalize Everything

Data Structure	Special Case of Graph
Array	Path graph (each node connects to the next)
Linked List	Directed path graph
Tree	Connected, acyclic, undirected graph ($n-1$ edges)
Binary Tree	Tree with max degree 3 (2 children + 1 parent)
DAG	Directed Acyclic Graph (build dependencies)
Social Network	Undirected graph with weighted edges
Road Map	Weighted undirected graph
Internet	Directed weighted graph (BGP routing)

Key Terminology

Term	Definition	Analogy
Vertex	Element of V	City, user, server
Edge	Element of E — pair (u,v)	Road, friendship, link
Directed	$(u \rightarrow v) \neq (v \rightarrow u)$	One-way street
Undirected	$\{u,v\} = \{v,u\}$	Two-way road
Weighted	Edge carries numeric weight $w(u,v)$	Road distance, cost
Degree	Edges incident to a vertex	Number of connections
In-degree	Edges pointing INTO a vertex	Followers count
Out-degree	Edges pointing OUT of a vertex	Following count
DAG	Directed Acyclic Graph	Build systems, spreadsheets
Simple Graph	No self-loops, no multi-edges	Most algorithm problems

Fundamental Properties



Handshaking Lemma: Sum of all degrees = $2 \times |E|$.
Consequence: number of odd-degree vertices is always EVEN.



Edge Bounds: Undirected simple graph: $0 \leq |E| \leq |V|(|V|-1)/2$. Tree: $|E| = |V| - 1$ exactly.

Why Graphs Exist

Section 3: Problems Graphs Solve That No Other Structure Can

Modeling Pairwise Relationships

Trees impose strict parent-child hierarchy. Graphs allow arbitrary pairwise relationships — a city can connect to any number of other cities directly, with no notion of "parent."

Reachability and Path Finding

"Can service A reach database B through the network?" requires graph traversal. No linear or hierarchical structure answers this generically.

Optimization Over Networks

"What is the cheapest route from A to B?" (Dijkstra), "What is the minimum cost to connect all offices?" (Prim/Kruskal) — inherently graph problems.

Dependency and Ordering

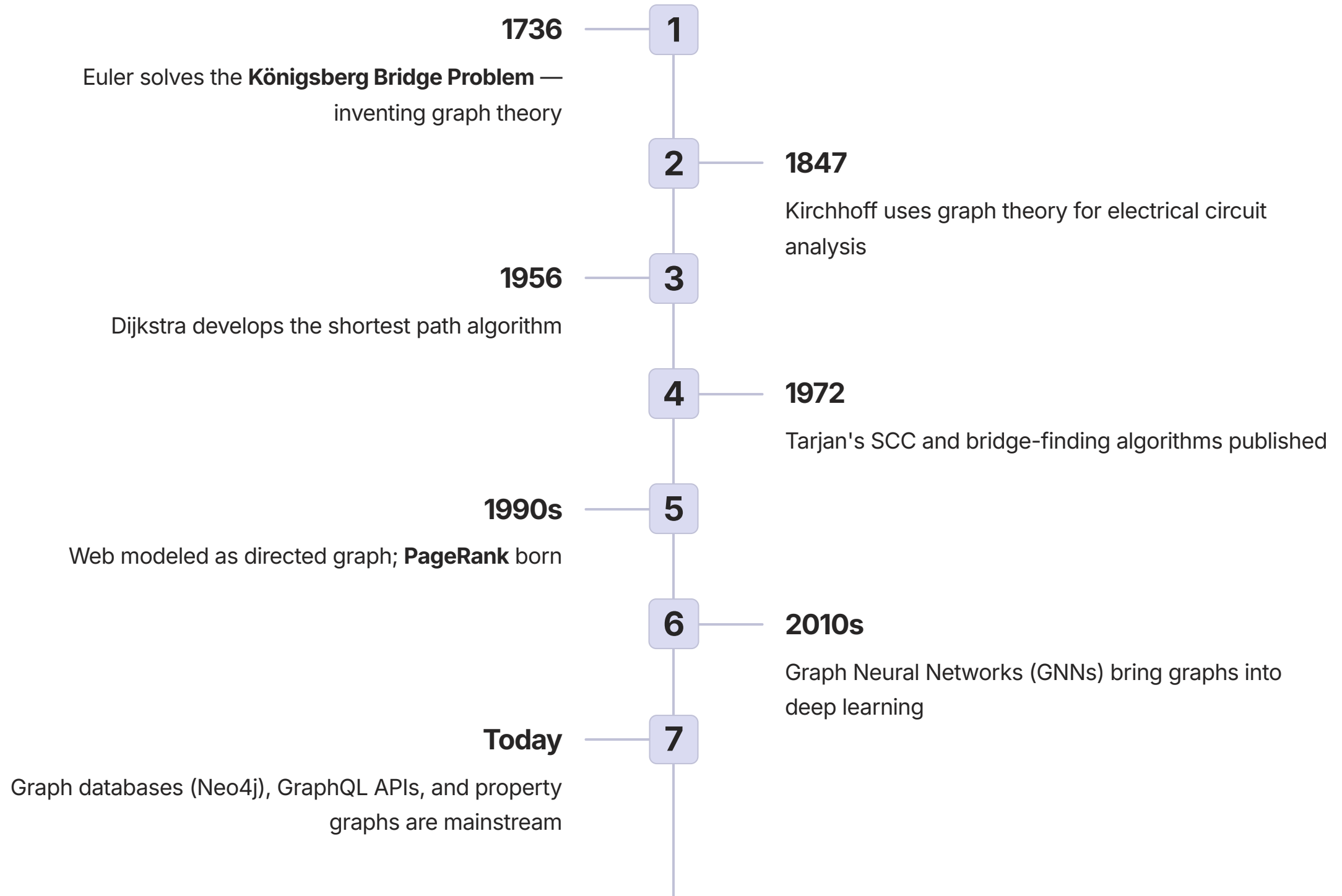
"In what order should I build these packages?" requires detecting cycles and computing topological order — only meaningful on a DAG.

Industry Mapping

Industry	Graph Problem	Algorithm
Maps / GPS	Shortest path between locations	Dijkstra, A*
Social Networks	Community detection, friend suggestions	BFS, Connected Components
Package Managers	Dependency resolution, install order	Topological Sort (Kahn's)
Compilers	Register allocation, dead code elimination	Graph Coloring, DFS
Networking	Routing table computation	Bellman-Ford, OSPF
Fraud Detection	Anomalous transaction patterns	Graph pattern matching
OS Scheduling	Deadlock detection	Cycle detection in resource graphs

Evolution & History of Graph Theory

Section 4: From Euler's Bridges to Graph Neural Networks



Graph Representations

Section 5: Representation 1 — Adjacency List

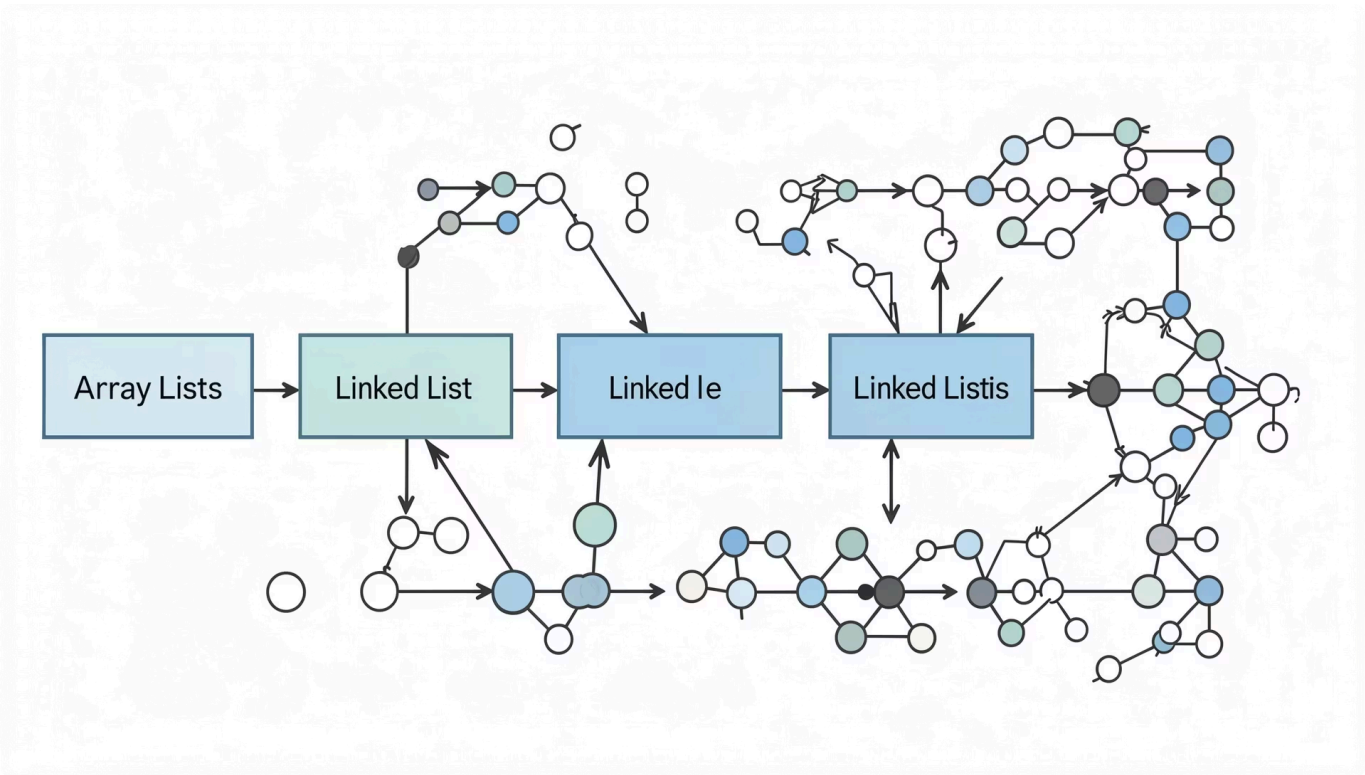
Structure: An array (or hash map) of size $|V|$, where each entry stores the list of neighbors.

Adjacency List:

- 0: [1, 2]
- 1: [0, 3]
- 2: [0, 3, 4]
- 3: [1, 2]
- 4: [2]

Directed Weighted:

- 0: [(1, 5), (2, 3)]
- 1: [(3, 1)]
- 2: [(3, 7), (4, 2)]



Complexity

Operation	Cost
Space	$O(V + E)$ ✓
Add edge	$O(1)$ amortized
Check edge	$O(\text{degree}(u))$
Get neighbors	$O(1)$
Iterate all	$O(V + E)$

```
CLASS Graph:
    adj = array of empty lists, size V

FUNCTION add_edge_undirected(u, v):
    adj[u].append(v)
    adj[v].append(u)

FUNCTION add_edge_weighted(u, v, w):
    adj[u].append((v, w))
    adj[v].append((u, w)) // Remove second line for directed

FUNCTION neighbors(u):
    RETURN adj[u]
```

Adjacency Matrix Representation

Section 5: Representation 2

Structure: A $|V| \times |V|$ matrix where `matrix[u][v] = 1` (or weight) if edge (u,v) exists, else 0.

Adjacency Matrix (0-indexed):

```
0 1 2 3
0 [0 1 1 0]
1 [1 0 0 1]
2 [1 0 0 1]
3 [0 1 1 0]
```

Complexity

Operation	Cost
Space	$O(V^2)$ ❌
Add edge	$O(1)$
Check edge	$O(1)$ ✅
Get neighbors	$O(V)$ scan required
Iterate all	$O(V^2)$

CLASS Graph:

`matrix = V×V array, initialized to 0`

FUNCTION `add_edge(u, v, weight=1)`:

`matrix[u][v] = weight`

`matrix[v][u] = weight` // Remove for directed

FUNCTION `has_edge(u, v)`:

`RETURN matrix[u][v] != 0` // $O(1)$

FUNCTION `neighbors(u)`:

`result = []`

`FOR v = 0 TO V-1:`

`IF matrix[u][v] != 0:`

`result.append(v)`

`RETURN result` // $O(V)$ scan required

⚠️ Memory Warning: For $V = 100,000$ nodes, an adjacency matrix requires $100K^2 = 10^{10}$ entries $\approx 40GB$. Never use for sparse graphs with large V .

Edge List Representation

Section 5: Representation 3

Structure: A flat list of all edges as (u, v) or (u, v, w) tuples.

Graph as edge list:
[(0,1), (0,2), (1,3), (2,3)]

Weighted:
[(0,1,5), (0,2,3), (1,3,1), (2,3,7)]

Complexity

Operation	Cost
Space	$O(E)$ ✓
Add edge	$O(1)$ append
Check edge	$O(E)$ linear scan
Get neighbors	$O(E)$ must filter
Iterate all	$O(E)$

When to Use Edge List

Kruskal's MST

Sort edges by weight — edge list is the natural structure

Graph Serialization

Compact storage and transmission of graph data

Offline Processing

Single-pass algorithms that process all edges once

Choosing Your Representation

Comparison Matrix — The Core Engineering Decision

Operation	Adjacency List	Adjacency Matrix	Edge List
Space	$O(V+E)$ ✓	$O(V^2)$ ✗	$O(E)$ ✓
Add edge	$O(1)$	$O(1)$	$O(1)$
Check edge (u,v)	$O(\text{degree})$	$O(1)$ ✓	$O(E)$
Get all neighbors	$O(\text{degree})$ ✓	$O(V)$	$O(E)$
Iterate all edges	$O(V+E)$ ✓	$O(V^2)$	$O(E)$ ✓
Delete edge	$O(\text{degree})$	$O(1)$	$O(E)$
Best for	BFS, DFS, most algorithms	Dense graphs, quick edge check	Kruskal's MST, offline processing
Sparse graph?	✓ Use this	✗ Wastes memory	✓ Also good
Dense graph?	✓ Still fine	✓ Constant lookup wins	✗ Too many edges

👉 **Production Rule:** Default to adjacency list. Use adjacency matrix only when $V \leq 1000$ AND you need $O(1)$ edge existence checks. Use edge list for algorithms that process all edges sorted (Kruskal's).

Graph Classification & Connected Components

Connected Components

An undirected graph may consist of multiple **connected components** — groups of vertices where every vertex can reach every other within the group, but no edges exist between groups.

Graph:

```
0—1  3—4  6
|    \
2      5
```

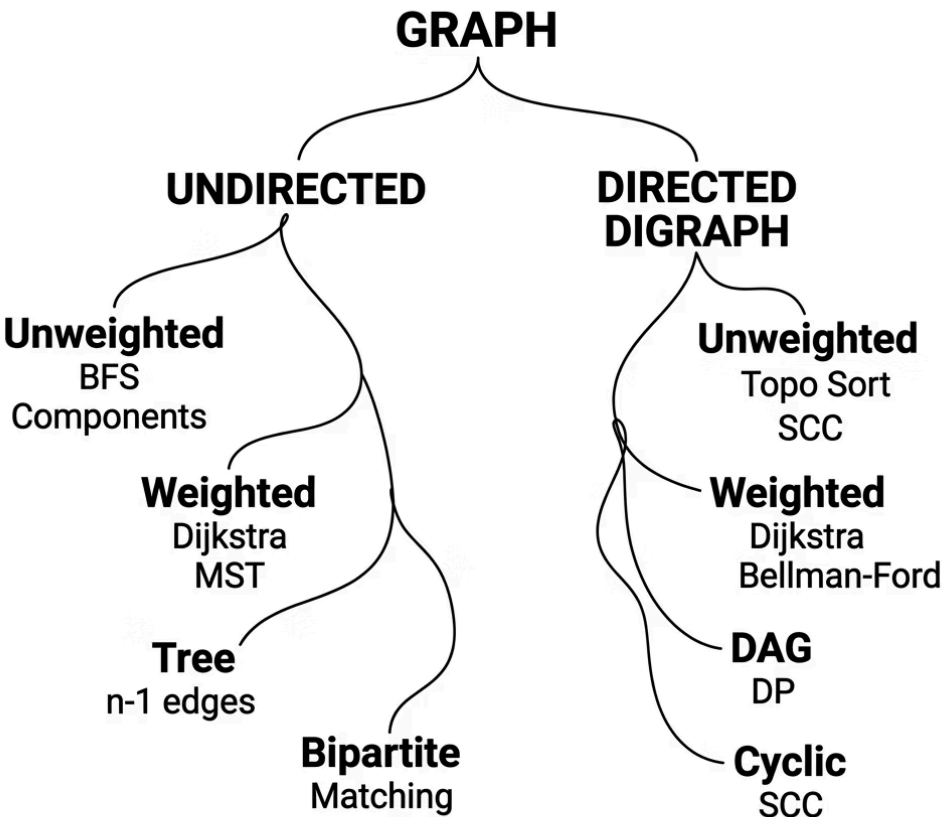
Components: {0,1,2}, {3,4,5}, {6}

Number of components: 3

i Finding all components: Run BFS/DFS from each unvisited vertex. Each new BFS/DFS invocation = a new component.

Bipartite Graph Detection

```
FUNCTION is_bipartite(graph, V):
    color = array of -1, size V
    FOR each unvisited vertex s:
        color[s] = 0
        queue = [s]
        WHILE queue not empty:
            u = dequeue(queue)
            FOR each neighbor v of u:
                IF color[v] == -1:
                    color[v] = 1 - color[u]
                    queue.enqueue(v)
                ELIF color[v] == color[u]:
                    RETURN FALSE
    RETURN TRUE
// Bipartite no odd-length cycle
```



Cycle Detection

Undirected Graph

A cycle exists if BFS/DFS encounters a visited vertex that is **NOT the parent**.

DFS **cycle** detection (undirected):

FUNCTION has_cycle_util(v, visited, parent):

visited[v] = **TRUE**

FOR each neighbor u **of** v:

IF **NOT** visited[u]:

IF has_cycle_util(u, visited, v):

RETURN TRUE

ELSE IF u != parent:

RETURN TRUE // visited non-parent = **cycle**

RETURN FALSE

Directed Graph

A cycle exists if DFS encounters a vertex in the **current recursion stack** — not just visited.

FUNCTION has_cycle_directed(v, visited, rec_stack):

visited[v] = **TRUE**

rec_stack[v] = **TRUE**

FOR each neighbor u **of** v:

IF **NOT** visited[u]:

IF has_cycle_directed(u, visited, rec_stack):

RETURN TRUE

ELIF rec_stack[u]:

RETURN TRUE // in **CURRENT** path = back edge = **cycle**

rec_stack[v] = **FALSE** // backtrack

RETURN FALSE

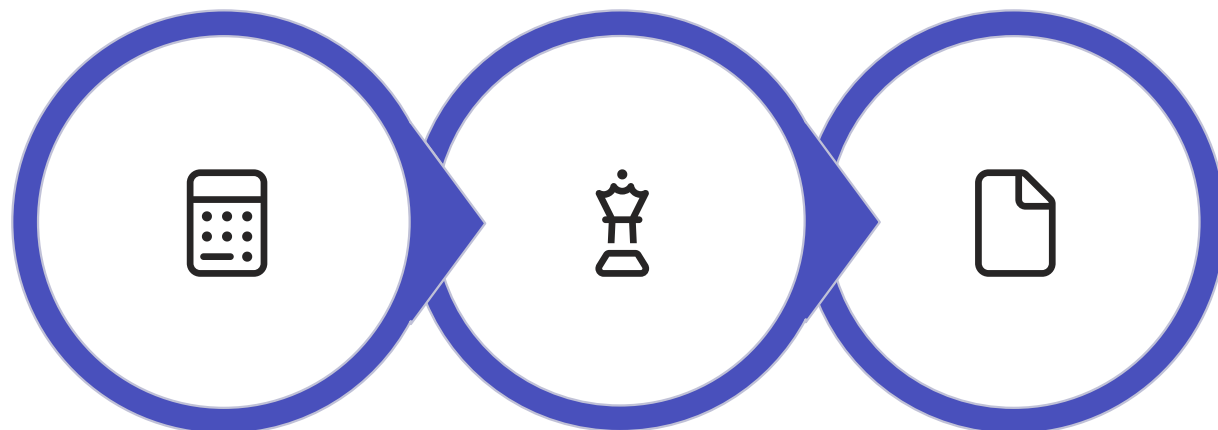


Critical: For directed graphs, **visited** alone is insufficient. You **MUST** maintain a separate **rec_stack** to track the current DFS path.

Topological Sort

Kahn's Algorithm (BFS-based) — DAG Only

A **topological ordering** of a DAG is a linear ordering of vertices such that for every directed edge $(u \rightarrow v)$, u appears before v .
Applications: build order for packages, course prerequisites, task scheduling.



Compute In-Degrees

Enqueue Zero Sources

BFS Process

```
FUNCTION topological_sort(graph, V):
    in_degree = array of 0s, size V
    FOR each vertex u:
        FOR each neighbor v of u:
            in_degree[v] += 1

    queue = all vertices with in_degree == 0 // Sources: no dependencies
    result = []

    WHILE queue not empty:
        u = dequeue(queue)
        result.append(u)
        FOR each neighbor v of u:
            in_degree[v] -= 1
            IF in_degree[v] == 0:
                queue.enqueue(v)

    IF len(result) != V:
        RETURN "Cycle detected — no topological order exists"
    RETURN result

// Time: O(V + E) Space: O(V)
// If result.size < V → cycle exists (not a DAG)
```

🟢 **Free Cycle Detection:** If `result.size < V`, a cycle exists. Kahn's gives you cycle detection as a by-product at no extra cost.

Special Graph Types

Section 6: Variants and Their Properties

Graph Type	Definition	Key Property	Canonical Algorithm
Complete Graph (K_n)	Every pair of vertices connected	$ E = n(n-1)/2$	Dense algorithm variants
Bipartite Graph	Vertices split into 2 sets; edges only cross sets	2-colorable; no odd cycles	Bipartite matching (Hopcroft-Karp)
Tree	Connected, acyclic undirected graph	$ E = V - 1$	DFS/BFS trivially
DAG	Directed, no cycles	Topological order exists	Topo sort, DP on DAG
Planar Graph	Drawable in plane without edge crossings	$ E \leq 3 V - 6$	Complex; rare in interviews
Eulerian Graph	Path/circuit using every edge exactly once	All degrees even (circuit)	Hierholzer's algorithm
Hamiltonian Graph	Path/circuit visiting every vertex once	NP-complete in general	Backtracking

Use Case 1: Dependency Resolution

Section 7: Package Managers (npm, pip, apt)

The Problem

User runs `npm install express`. Express depends on A, B, C; A depends on D; D depends on E. What is the correct install order? Is there a circular dependency?

The Solution

Model packages as vertices; `depends_on` as directed edges → DAG. Run topological sort (Kahn's) for install order; cycle detection for circular dependency error.

- ✅ **Why Graph?** No other structure represents arbitrary dependency relationships with efficient cycle detection and ordering.

Graph (directed):

`express` → [`accepts`, `debug`, `finalhandler`]

`debug` → [`ms`]

`ms` → []

Topological order:

`ms` → `debug` → `accepts`

→ `finalhandler` → `express`

If $A \rightarrow B \rightarrow A$ exists:

CIRCULAR DEPENDENCY ERROR

← exactly what npm reports

- ⚠️ **Production Parallel:** This is exactly how `npm install`, `pip install`, and `apt-get install` resolve package dependency chains.

Use Case 2: Social Network Friend Suggestions

Facebook, LinkedIn — Degrees of Separation

The Problem

Find all users within 2 degrees of separation from user U.
Suggest friends-of-friends not already connected to U.

The Solution

Undirected graph (users = vertices, friendships = edges). BFS from U for depth 2. Friend suggestions = vertices at depth exactly 2 not already adjacent to U.

BFS from Alice (depth 2):

Depth 1: Bob, Carol (direct friends)

Depth 2: Dave, Eve, Frank

Suggestions = {Dave, Eve, Frank}
- {Alice's existing friends}

Scale Insight: Facebook's graph has ~3 billion vertices and hundreds of billions of edges. Their internal system (TAO) is purpose-built for social graph queries.

Vertices

~3 billion users

Edges

Hundreds of billions of friendships

System

TAO — distributed social graph store

Use Case 3: Network Routing — OSPF Protocol

Networking / Telecommunications

The Problem

Each router must compute the shortest path to every other router in the network autonomously and update when topology changes.

The Solution

Each router maintains a weighted directed graph of the network (weights = link costs). Dijkstra's algorithm computes its routing table. Link-state advertisements (LSAs) propagate topology changes via BFS-like flooding.

Network as weighted graph:

Router A —(5)— Router B —(3)— Router C
 \ —(2)— Router D —(1)— Router C

Shortest A→C:

A→D→C (cost 3)

NOT A→B→C (cost 8)

Dijkstra computes this in

$O((V+E) \log V)$

on the router's local topology view

- ✓ Dijkstra's algorithm runs live on every router in the internet's OSPF protocol — graph theory powering global connectivity.

Use Case 4: Deadlock Detection in OS

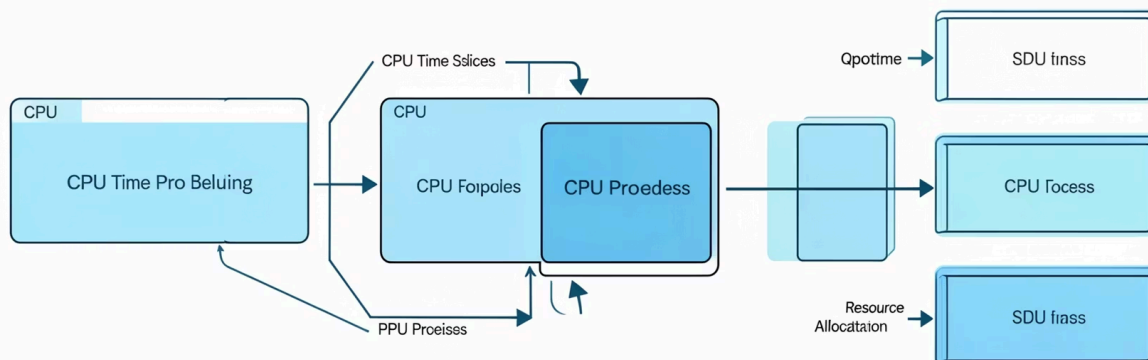
Resource Allocation Graph (RAG)

The Problem

Multiple processes compete for resources. Detect if a deadlock exists (circular wait condition).

The Solution

Resource Allocation Graph (RAG).
Processes and resources as vertices.
Assignment edges (resource → process)
and request edges (process → resource).
A cycle in this graph = deadlock.



P1 holds R1, requests R2

P2 holds R2, requests R1

Graph:

$P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$

← Cycle detected = DEADLOCK

⊗ **Real Impact:** Undetected deadlocks freeze entire systems. OS kernels run cycle detection on the RAG continuously to prevent this.

Implementation 1: Complete Graph Class

Section 8: Adjacency List — Production Quality

CLASS Graph:

V = 0

adj = {} // vertex → list of (neighbor, weight)

FUNCTION __init__(V):

self.V = V

FOR i = 0 TO V-1:

self.adj[i] = []

FUNCTION add_edge(u, v, weight=1, directed=FALSE):

self.adj[u].append((v, weight))

IF NOT directed:

self.adj[v].append((u, weight))

FUNCTION neighbors(u):

RETURN self.adj[u]

FUNCTION degree(u):

RETURN len(self.adj[u])

FUNCTION has_edge(u, v):

FOR (neighbor, w) in self.adj[u]:

IF neighbor == v: RETURN TRUE

RETURN FALSE

FUNCTION print_graph():

FOR u = 0 TO V-1:

PRINT u, "→", self.adj[u]

// Usage:

g = Graph(5)

g.add_edge(0, 1)

g.add_edge(0, 2)

g.add_edge(1, 3, directed=TRUE)

g.add_edge(2, 4, weight=7)

Implementation 2: Connected Components

Iterative DFS — $O(V + E)$

```
FUNCTION connected_components(graph, V):  
    visited = array of FALSE, size V  
    components = []  
  
    FOR u = 0 TO V-1:  
        IF NOT visited[u]:  
            // Start a new component  
            component = []  
            stack = [u]  
            WHILE stack not empty:  
                v = stack.pop()  
                IF visited[v]: CONTINUE  
                visited[v] = TRUE  
                component.append(v)  
                FOR (neighbor, _) in graph.neighbors(v):  
                    IF NOT visited[neighbor]:  
                        stack.append(neighbor)  
            components.append(component)  
  
    RETURN components  
  
// Returns list of lists; each inner list = one component  
// Time:  $O(V + E)$  Space:  $O(V)$ 
```

 **Key Pattern:** The outer `FOR u = 0 TO V-1` loop ensures disconnected graphs are handled correctly. Never assume the graph is connected.

Implementation 3: Directed Cycle Detection

DFS with rec_stack — The Critical Distinction

```
FUNCTION has_cycle_directed(graph, V):  
    visited = array of FALSE, size V  
    rec_stack = array of FALSE, size V  
  
    FUNCTION dfs(u):  
        visited[u] = TRUE  
        rec_stack[u] = TRUE  
  
        FOR (v, _) in graph.neighbors(u):  
            IF NOT visited[v]:  
                IF dfs(v): RETURN TRUE  
            ELIF rec_stack[v]:  
                RETURN TRUE    // Back edge in DFS = cycle  
  
        rec_stack[u] = FALSE    // Done with this path — backtrack  
        RETURN FALSE  
  
    FOR u = 0 TO V-1:  
        IF NOT visited[u]:  
            IF dfs(u): RETURN TRUE  
    RETURN FALSE
```

visited[v] = TRUE

We have *ever* visited vertex v during any DFS traversal

rec_stack[v] = TRUE

Vertex v is in the *current active* DFS path — a back edge here means a cycle

Implementation 4: Topological Sort — Kahn's

BFS-Based — Production Preferred

```
FUNCTION topological_sort(graph, V):
    in_degree = array of 0s, size V
    FOR u = 0 TO V-1:
        FOR (v, _) in graph.neighbors(u):
            in_degree[v] += 1

    queue = deque()
    FOR u = 0 TO V-1:
        IF in_degree[u] == 0:
            queue.append(u)

    result = []
    WHILE queue not empty:
        u = queue.popleft()
        result.append(u)
        FOR (v, _) in graph.neighbors(u):
            in_degree[v] -= 1
            IF in_degree[v] == 0:
                queue.append(v)

    IF len(result) < V:
        RETURN ERROR "Graph has a cycle — topological sort impossible"
    RETURN result

// Classic production use: Kahn's is preferred over DFS-based topo sort
// because cycle detection is a FREE by-product (result.size < V → cycle)
```

✅ **Why Kahn's over DFS-based?** Cycle detection is free — if `result.size < V`, a cycle exists. No extra bookkeeping needed.

Lab 1: Build and Analyze a Graph

Section 9: Hands-On Lab — Intermediate/Advanced

Graph Specification

Vertices: 0–6

Edges (undirected):

(0,1), (0,2), (1,3),

(2,3), (4,5)

Vertex 6: isolated

Expected:

Components: {0,1,2,3}, {4,5}, {6}

→ 3 components

Degrees:

vertex 0→2, 1→2, 2→2, 3→2

vertex 4→1, 5→1, 6→0

Handshaking:

$\sum \text{degrees} = 2 \times |E| = 2 \times 5 = 10 \checkmark$

Tasks

01

Build the Graph

Implement using the Graph class with adjacency list

02

Component Detection

Implement `connected_components` and verify the 3 components

03

Verify Handshaking Lemma

Sum of all degrees = $2 \times$ edge count

04

Add Edge (3,0)

Does it create a multi-edge? Handle gracefully.

Lab 2: DAG and Topological Sort

Course Prerequisite System

Course Graph (Directed Edges)

Courses: Math, Physics, CS101,
CS201, CS301, Algorithms

Prerequisites:

Math → Physics

Math → CS101

Physics → CS201

CS101 → CS201

CS201 → CS301

CS301 → Algorithms

Valid order:

Math → Physics → CS101

→ CS201 → CS301

→ Algorithms

Production Parallel: This is exactly how `npm install`, `pip install`, and `apt-get install` resolve package dependency chains.

Tasks

01

Build Directed Graph

Adjacency list with course names mapped to indices via hash map

02

Run Kahn's Topo Sort

Find a valid course enrollment order

03

Inject Artificial Cycle

Add `Algorithms → Math` and re-run; confirm cycle detection triggers

04

Remove Cycle

Confirm topological sort completes successfully

Performance Considerations

Section 11: Representation Choice Impact on Algorithm Performance

Algorithm	Best Representation	Reason
BFS / DFS	Adjacency List	$O(V+E)$ — iterate only real edges
Dijkstra (sparse)	Adj. List + Heap	$O((V+E) \log V)$ with min-heap
Dijkstra (dense)	Adjacency Matrix	$O(V^2)$ — matrix lookup faster
Floyd-Warshall	Adjacency Matrix	$O(V^3)$ — requires all-pairs access
Kruskal's MST	Edge List (sorted)	$O(E \log E)$ — sort all edges once
Prim's MST	Adj. List + Heap	$O((V+E) \log V)$ for sparse
Topological Sort	Adjacency List	$O(V+E)$ — Kahn's or DFS
SCC (Tarjan/Kosaraju)	Adjacency List	$O(V+E)$

 **Memory Bottleneck:** Sparse graph with 1M nodes: Adjacency List = $O(V)$ space. Adjacency Matrix = $O(V^2) = 10^{12}$ entries = impossible. Real-world social graphs are stored as distributed adjacency lists.

Scalability & Large-Scale Graph Systems

Section 12: Production Graph Infrastructure



Facebook TAO

Billions of vertices, trillions of edges. Distributed adjacency list, sharded by user ID.



Google Maps

300M+ road segments. Hierarchical graph with precomputed shortcuts (Contraction Hierarchies).



Neo4j

Billions of nodes/edges. Native graph storage with index-free adjacency.



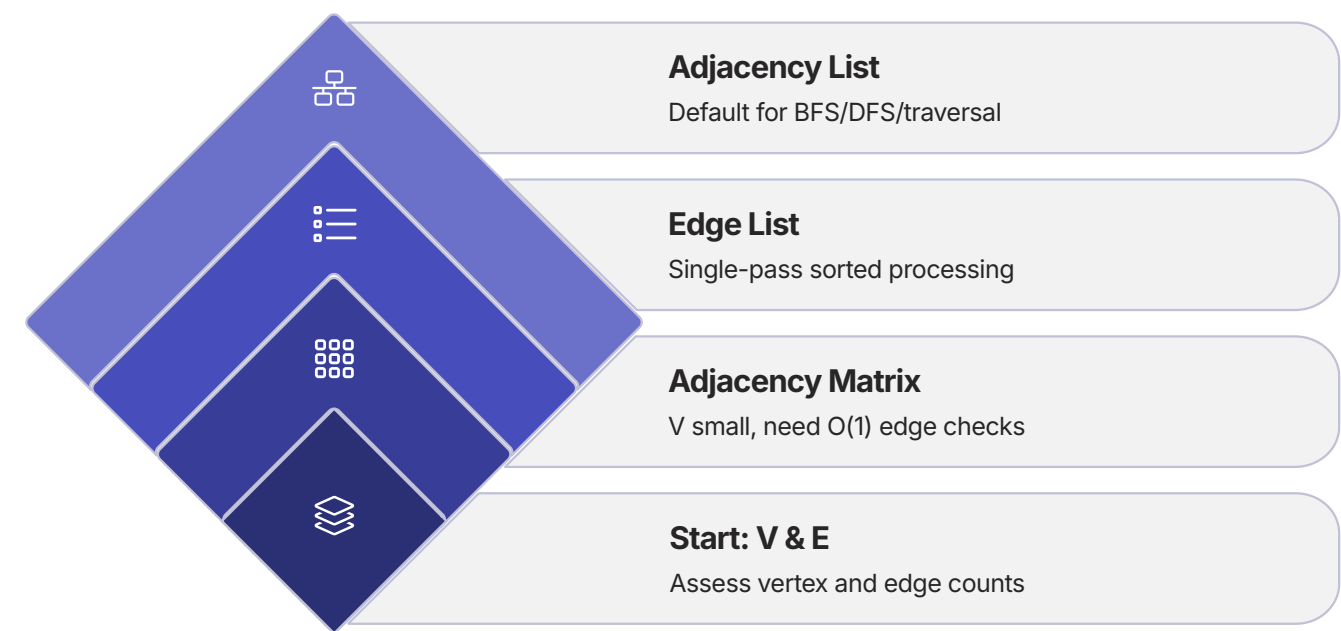
Apache Giraph / Pregel

Trillion-edge graphs. BSP (Bulk Synchronous Parallel) graph processing.

ⓘ Distributed Partitioning Challenge: Edges crossing partition boundaries = expensive network calls. Goal: minimize edge-cuts while keeping partitions balanced. Methods range from simple vertex hashing to community-aware partitioning.

Design & Architectural Considerations

Section 13: Representation Selection Framework



When to Use Implicit Graphs

Many problems don't explicitly give you a graph — you derive it from the problem structure.

Implicit graphs are covered in depth in Part 2: Matrix as Graph.

2D Grid

Cells are vertices, adjacent cells are edges

Word Ladder

Words are vertices, single-character changes are edges

Number States

Numbers are vertices, valid transitions are edges

Best Practices

Section 15: Do's and Don'ts

✓ DO'S

Category	Practice
Design	Always clarify: directed or undirected? weighted? cycles possible? simple graph?
Design	Default to adjacency list unless you have a specific reason for matrix/edge list
Implementation	Use 0-indexed vertices internally; map labels to indices via a hash map
Implementation	Handle disconnected graphs — always iterate over ALL vertices for BFS/DFS
Performance	For Dijkstra on sparse graphs, use a priority queue (min-heap), not a linear scan
Correctness	In directed cycle detection: maintain a <code>rec_stack</code> separate from <code>visited</code>
Testing	Test on: empty graph, single vertex, disconnected graph, self-loops, dense complete graph

✗ DON'TS

Category	Anti-Pattern
Design	Don't use adjacency matrix for sparse graphs with $V > 10,000$ — memory blows up
Implementation	Don't forget to start BFS/DFS from every unvisited vertex — disconnected graphs exist
Correctness	Don't use only <code>visited</code> for directed cycle detection — use <code>rec_stack</code>
Performance	Don't run $O(V^2)$ Dijkstra on sparse graphs — use a heap for $O((V+E) \log V)$
Design	Don't assume a graph is connected without checking — always verify

Common Mistakes & Pitfalls

Section 16: Errors by Level

Level	Mistake	Why	Impact	Fix
Intermediate	Using visited alone for directed cycle detection	Works for undirected, not directed	False negatives — misses directed cycles	Use rec_stack to track current DFS path
Intermediate	BFS/DFS only from vertex 0	Assuming connected graph	Misses entire components	Loop: FOR u in V: IF NOT visited[u]: BFS/DFS(u)
Intermediate	Adjacency matrix for large sparse graph	Didn't estimate memory	OOM crash	Estimate: $V=100K \rightarrow 100K^2 = 10^{10}$ entries $\approx$ 40GB
Advanced	Not handling negative weights in Dijkstra	Dijkstra assumes non-negative	Wrong shortest paths	Use Bellman-Ford for negative weights
Advanced	Modifying graph during traversal	Seems harmless	Iterator invalidation / wrong results	Clone neighbor list before iteration if mutations possible
Advanced	Off-by-one in in-degree initialization for Kahn's	Forgetting to count all incoming edges	Incorrect topological sort	Build in-degree from all edges; double-check with edge count

Comparison with Alternatives

Section 17: Graph vs. Other Data Structures

Feature	Graph (Adj. List)	Graph (Adj. Matrix)	Tree	Hash Table
Arbitrary relationships	✓	✓	✗ hierarchical only	✗
Cycle support	✓	✓	✗	N/A
Space (sparse)	$O(V+E)$ ✓	$O(V^2)$ ✗	$O(V)$	$O(n)$
Edge check	$O(\text{degree})$	$O(1)$ ✓	$O(\log n)$ path	$O(1)$ avg
Traversal	$O(V+E)$	$O(V^2)$	$O(n)$	N/A
Shortest path	✓ Dijkstra	✓ Floyd	Limited	✗
Ordered structure	✗ use topo sort	✗	✓ BST	✗
Best for	Network, social, maps	Dense, small graphs	Hierarchy, ordering	Key-value lookup

Graph Basics — Learning Summary

Section 18: Key Takeaways & Quick Reference

1 **$G = (V, E)$ is the most general relational data structure**

Trees, lists, and arrays are all special cases of graphs.

2 **Three representations with distinct trade-offs**

Adjacency List (default), Adjacency Matrix (dense/small), Edge List (Kruskal's, offline).

3 **Directed cycle detection requires `rec_stack`**

`visited` alone is insufficient for directed graphs — you must track the current DFS path.

4 **Topological sort (Kahn's) is $O(V+E)$ with free cycle detection**

If `result.size < V`, a cycle exists — no extra work needed.

5 **Connected components: loop ALL vertices**

Run BFS/DFS from every unvisited vertex; each invocation = one component.

Common Interview Questions

1. How do you detect a cycle in a directed vs. undirected graph? Why are they different?
2. What is the difference between adjacency list and adjacency matrix? When do you use each?
3. How does Kahn's algorithm detect cycles as a free by-product?
4. What is the maximum number of edges in a simple undirected graph with V vertices?
5. How do you find all connected components in $O(V+E)$?
6. What properties must a graph have for a topological sort to exist?
7. In a directed graph, what is the difference between strongly connected and weakly connected?
8. How would you represent a weighted directed graph in memory for Dijkstra's algorithm?
9. Given a dependency graph, how do you detect a circular dependency?

PART 2: Matrix as Graph

Section 1: Topic Classification

Field	Details
Topic Name	Matrix as Graph
Category	Algorithms / Problem Solving Patterns
Domain / Subdomain	Computer Science / Graph Algorithms / Implicit Graphs
Topic Type	Pattern-Based + Algorithmic + Implementation
Difficulty Level	Advanced
Prerequisites	Graph Basics, BFS, DFS, 2D Arrays, Recursion
Industry Relevance	Very High — Core pattern in robotics path planning, image processing, game engines, flood-fill algorithms, maze solving, geographic mapping
Recommended Learning Path	Graph Basics → BFS → DFS → Matrix as Graph → Shortest Path on Grid → Advanced Grid DP

What Is Matrix as Graph?

Section 2: The Core Insight

i A 2D matrix (grid) is an **implicit graph** where each cell (row, col) is a **vertex**, two cells are connected by an **edge** if they are adjacent, and cell values determine **traversability**.

The graph is called "implicit" because **you never actually build an adjacency list**. Instead, you derive neighbors on-the-fly from the grid coordinates using a directional array.

Matrix:

```
[ 1 1 0 1 ]
[ 1 0 0 1 ]
[ 1 1 1 1 ]
[ 0 0 1 0 ]
```

Implicit Graph (1=passable, 0=wall):

```
(0,0)---(0,1)    (0,3)
 |          |
(1,0)      (1,3)
 |          |
(2,0)---(2,1)---(2,2)---(2,3)
                |
                (3,2)
```

Why This Pattern Is Fundamental

Matrix-as-graph is arguably the **single most frequently tested graph pattern** in technical interviews at FAANG-tier companies.

Real-World Mappings

Real-World Problem	Grid Interpretation
GPS routing on street grid	Cells = intersections; edges = streets
Image flood fill (paint bucket)	Pixels = vertices; same-color adjacency = edges
Robotic path planning	Grid = map; walls = impassable vertices
Game enemy pathfinding	Tiles = vertices; walkable adjacency = edges
Wildfire spread simulation	Cells = land patches; adjacency = fire spread
Maze solving	Cells = vertices; open passages = edges

Why Matrix as Graph Exists

Section 3: The Problem Without This Pattern

Without the Pattern

"Find shortest path from top-left to bottom-right" → confusion about what algorithm applies. "Count islands in a grid" → unclear structure to traverse. "Flood fill a region" → unknown traversal strategy.

With the Pattern

Shortest path → BFS on the implicit graph. Count islands → connected components via BFS/DFS. Flood fill → DFS marking visited cells. Spread simulation → multi-source BFS.



Robotics

Amazon Robotics warehouse navigation — the warehouse floor IS the grid graph



Medical Imaging

Tumor detection in MRI — connected regions of abnormal pixel intensity



Emergency Management

Wildfire spread prediction — multi-source BFS from initial fire cells



Game Development

Enemy pathfinding on tile maps — A* on the implicit grid graph

The Direction Array — Core Building Block

Section 5: How It Works

 Instead of hard-coding neighbor logic, use a **direction array** to iterate over neighbors generically. This is the universal building block for all grid algorithms.

 **Non-Negotiable Rule:** The bounds check `0 ≤ nr < ROWS AND 0 ≤ nc < COLS` must appear before EVERY neighbor access. Forgetting it causes out-of-bounds crashes — the #1 bug in grid problems.

```
// 4-directional (most common)
DIRECTIONS_4 = [(-1,0), (1,0), (0,-1), (0,1)]

// 8-directional (includes diagonals)
DIRECTIONS_8 = [(-1,-1), (-1,0), (-1,1),
                (0,-1),    (0,1),
                (1,-1), (1,0), (1,1)]

// Get valid neighbors of cell (r, c):
FUNCTION get_neighbors(r, c, grid, directions):
    neighbors = []
    FOR (dr, dc) in directions:
        nr = r + dr
        nc = c + dc
        IF 0 ≤ nr < ROWS AND 0 ≤ nc < COLS:
            IF grid[nr][nc] is passable:
                neighbors.append((nr, nc))
    RETURN neighbors
```

4-Directional

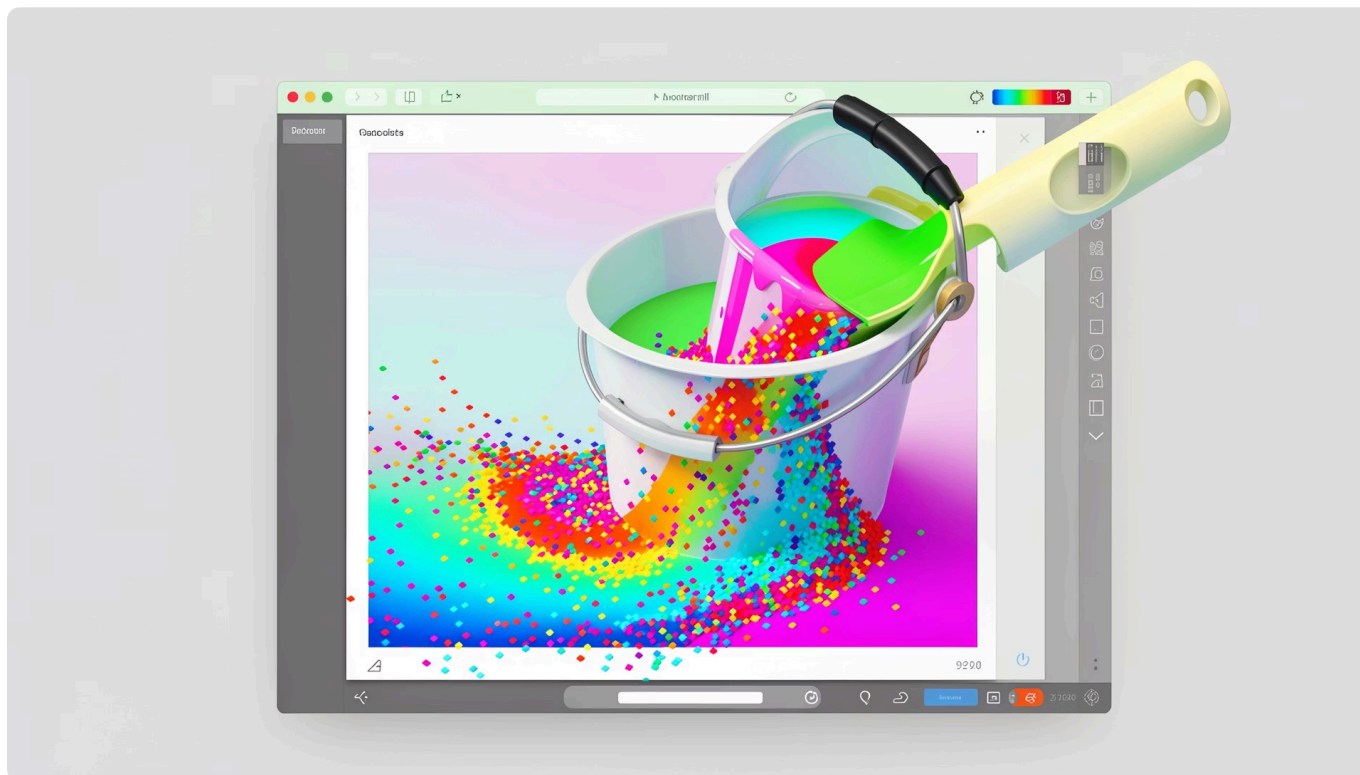
Up, Down, Left, Right —
most grid problems

8-Directional

Includes diagonals —
chess, image processing

Pattern 1: Flood Fill / DFS Marking

Image Paint-Bucket Fill, Island Marking, Region Clearing



```
FUNCTION flood_fill(grid, r, c,
original_val, new_val):
    // Base cases
    IF r < 0 OR r >= ROWS OR c < 0 OR c
>= COLS:
        RETURN
    IF grid[r][c] != original_val:
        RETURN // Different value or
already filled

    grid[r][c] = new_val // Mark / fill

    // Recurse in 4 directions
    flood_fill(grid, r-1, c, original_val,
new_val)
    flood_fill(grid, r+1, c, original_val,
new_val)
    flood_fill(grid, r, c-1, original_val,
new_val)
    flood_fill(grid, r, c+1, original_val,
new_val)

// Example:
// Grid: 1 1 0    After
flood_fill(0,0,1,2):
//   1 0 0    Grid: 2 2 0
//   1 1 1      2 0 0
//               2 2 2
```

✓ **Application:** Image paint-bucket fill, clearing visited regions, island marking. The simplest and most intuitive grid traversal pattern.

Pattern 2: Count Connected Components (Islands)

LeetCode #200 — Number of Islands

```
FUNCTION count_islands(grid):
  IF grid is empty: RETURN 0
  ROWS = grid.rows; COLS = grid.cols
  count = 0

  FOR r = 0 TO ROWS-1:
    FOR c = 0 TO COLS-1:
      IF grid[r][c] == 1:      // Found an unvisited land cell
        count += 1
        dfs_sink(grid, r, c)  // Sink the entire island

  RETURN count

FUNCTION dfs_sink(grid, r, c):
  IF r < 0 OR r >= ROWS OR c < 0 OR c >= COLS: RETURN
  IF grid[r][c] != 1: RETURN    // Water or already visited
  grid[r][c] = 0               // Sink — mark visited
  dfs_sink(grid, r-1, c)
  dfs_sink(grid, r+1, c)
  dfs_sink(grid, r, c-1)
  dfs_sink(grid, r, c+1)
```

O(N)

Time Complexity

Every cell visited at most once — once sunk, never revisited

O(N)

Space Complexity

Recursion stack in worst case (all 1s, snaking path)

 **Production Note:** For large grids, use iterative DFS/BFS to avoid stack overflow. Recursive DFS on a 10^6 -cell grid can exceed default stack limits.

Pattern 3: BFS for Shortest Path on Grid

Minimum Steps from Source to Destination

Key Insight: BFS on an unweighted graph finds the shortest path in terms of number of edges (steps). On a grid, each step moves to one adjacent cell.

Why BFS not DFS? BFS guarantees minimum steps. DFS finds *a* path, not the *shortest* path. This is the most common mistake in grid shortest-path problems.

```
FUNCTION bfs_shortest_path(grid, start, end):
    (sr, sc) = start; (er, ec) = end

    IF grid[sr][sc] == 1 OR grid[er][ec] == 1:
        RETURN -1 // Start or end is a wall

    visited = 2D array of FALSE
    queue = deque([(sr, sc, 0)])
    visited[sr][sc] = TRUE

    WHILE queue not empty:
        (r, c, dist) = queue.popleft()
        IF r == er AND c == ec:
            RETURN dist

        FOR (dr, dc) in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr = r + dr; nc = c + dc
            IF 0 <= nr < ROWS AND 0 <= nc < COLS:
                IF NOT visited[nr][nc] AND grid[nr][nc] == 0:
                    visited[nr][nc] = TRUE
                    queue.append((nr, nc, dist + 1))

    RETURN -1 // No path exists
```

Time

$O(\text{ROWS} \times \text{COLS})$

Space

$O(\text{ROWS} \times \text{COLS})$ —
visited array + queue

Pattern 4: Multi-Source BFS

Minimum Distance from ANY Source — Rotting Oranges, Walls & Gates

```
FUNCTION multi_source_bfs(grid, sources):
    queue = deque()
    visited = 2D array of FALSE
    dist = 2D array of INFINITY

    FOR each source (r, c) in sources:
        queue.append((r, c))
        visited[r][c] = TRUE
        dist[r][c] = 0

    WHILE queue not empty:
        (r, c) = queue.popleft()
        FOR (dr, dc) in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr = r + dr; nc = c + dc
            IF valid(nr, nc) AND NOT visited[nr][nc] AND passable(nr, nc):
                visited[nr][nc] = TRUE
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))

    RETURN dist
```

- ✔ **Critical Insight:** All sources start at distance 0 simultaneously. This is equivalent to adding a virtual super-source connected to all real sources. BFS wavefront expands from ALL sources at once → each cell gets minimum distance to any source.

Rotting Oranges Walkthrough (LeetCode #994)

Grid:

2	1	1
1	1	0
0	1	1

 (2=rotten, 1=fresh, 0=empty)

Minute 0: sources = [(0,0)]

Minute 1: (0,0) spreads to (0,1) and (1,0)

Minute 2: (0,1) spreads to (0,2); (1,0) spreads to (1,1)

Minute 3: (0,2) → (1,2); (1,1) → (2,1)

Minute 4: (1,2) → (2,2)

Total minutes: 4

Check: any remaining fresh orange? → No → return 4

Pattern 5: 0-1 BFS for Binary-Cost Grids

Free Edges (Cost 0) and Toll Edges (Cost 1) — $O(N)$ vs $O(N \log N)$

```
FUNCTION bfs_01(grid, start, end):
    dist = 2D array of INFINITY
    deque = double-ended queue
    dist[start.r][start.c] = 0
    deque.appendleft(start)

    WHILE deque not empty:
        (r, c) = deque.popleft()
        FOR each neighbor (nr, nc):
            cost = 1 IF grid[nr][nc] is toll ELSE 0
            new_dist = dist[r][c] + cost
            IF new_dist < dist[nr][nc]:
                dist[nr][nc] = new_dist
                IF cost == 0:
                    deque.appendleft((nr, nc)) // Free → front
                ELSE:
                    deque.append((nr, nc)) // Cost → back

    RETURN dist[end.r][end.c]
```

✓ **Why 0-1 BFS?** Runs in $O(V+E)$ — better than Dijkstra's $O((V+E) \log V)$ for binary-cost graphs. The deque acts as a priority queue specifically optimized for 0/1 costs.

Cost 0 Edge

Push to **front** of deque — processed immediately, no cost increase

Cost 1 Edge

Push to **back** of deque — processed after all free edges at current level

Pattern 6: Dijkstra on Grid

Matrix as Directed Weighted Graph — Variable Cell Entry Costs

```
FUNCTION dijkstra_grid(grid, start, end):
    ROWS = grid.rows; COLS = grid.cols
    dist = 2D array of INFINITY
    dist[start.r][start.c] = grid[start.r][start.c]

    min_heap = [(cost, r, c)] // (cost_to_reach, row, col)

    WHILE min_heap not empty:
        (cost, r, c) = heappop(min_heap)

        IF (r, c) == end: RETURN cost
        IF cost > dist[r][c]: CONTINUE // Stale entry — skip

        FOR (dr, dc) in DIRECTIONS_4:
            nr = r + dr; nc = c + dc
            IF valid(nr, nc):
                new_cost = dist[r][c] + grid[nr][nc]
                IF new_cost < dist[nr][nc]:
                    dist[nr][nc] = new_cost
                    heappush(min_heap, (new_cost, nr, nc))

    RETURN dist[end.r][end.c]

// Time: O(ROWS × COLS × log(ROWS × COLS))
// V = ROWS×COLS, log factor from heap operations
```

When to use Dijkstra on grid: Each cell has a different cost to enter (terrain costs, toll roads, energy consumption). Use 0-1 BFS instead if costs are only 0 or 1 — it's faster.

Visited Array Strategies

Critical Implementation Decision

Strategy 1: Separate visited[][] Array

- ✓ Preserves original grid
- ✓ Clear semantics
- ✗ Extra $O(\text{ROWS} \times \text{COLS})$ space

Strategy 2: Modify Grid In-Place (Sink)

- ✓ No extra space
- ✗ Modifies input (may be unacceptable)
- ✗ Requires restoration if backtracking needed

Strategy 3: Visited Set (Sparse Grids)

- ✓ Only stores visited cells
- ✓ Works for non-rectangular grids
- ✗ Hash overhead per lookup

- 🟢 **Production Rule:** Use separate `visited[][]` unless (a) the problem explicitly says you can modify input AND (b) no need to restore (one-pass problem).

The Mark-Before-Enqueue Rule

// WRONG — mark on dequeue:

```
queue.append((r, c))
```

// ... later ...

```
(r, c) = queue.popleft()
```

```
visited[r][c] = TRUE // Too late!
```

// CORRECT — mark on enqueue:

```
visited[r][c] = TRUE // Prevent re-enqueue
```

```
queue.append((r, c))
```

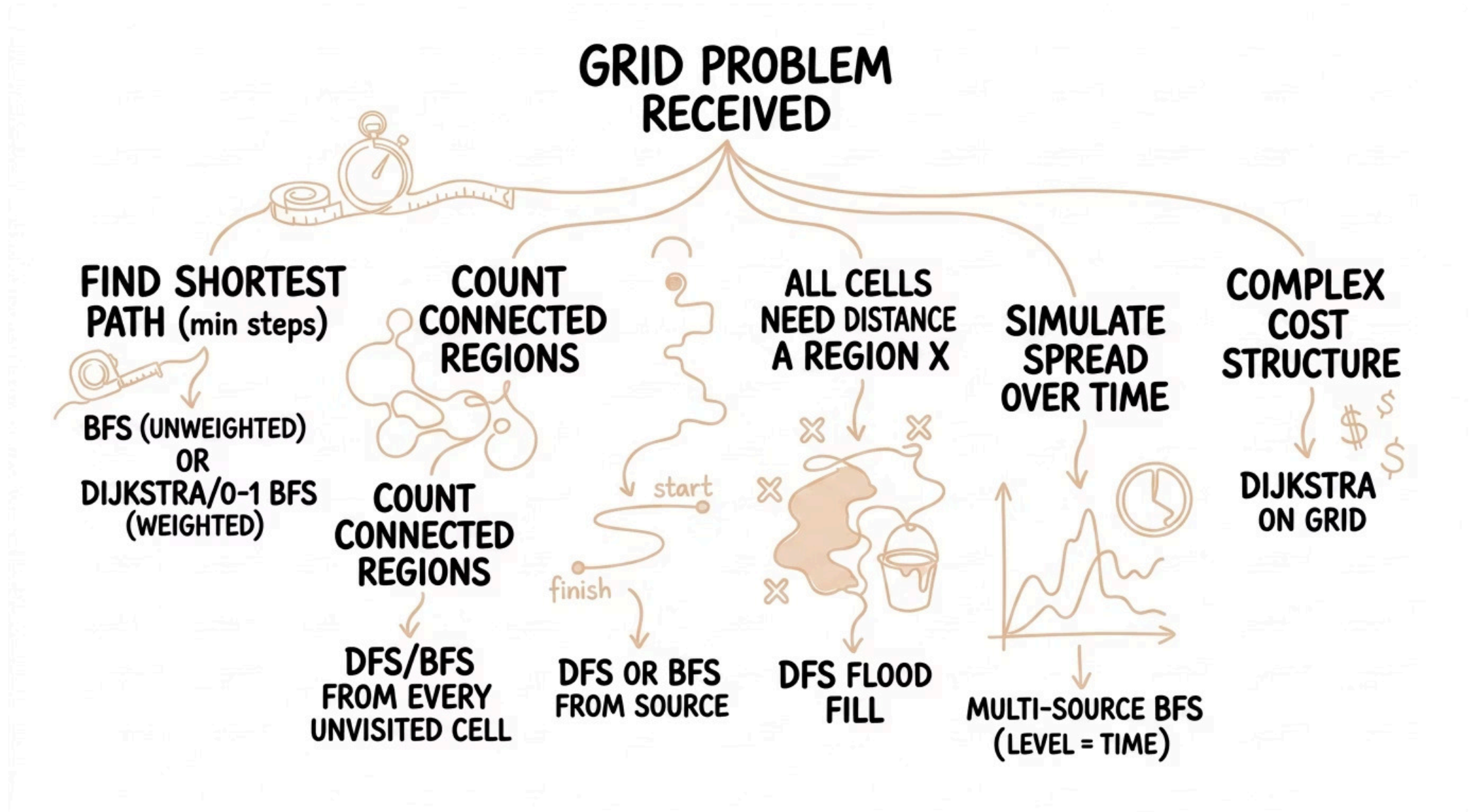
// Impact:

// Mark on dequeue: $O(N \times \text{branching})$ — cells enqueued multiple times

// Mark on enqueue: $O(N)$ — each cell enqueued exactly once

Grid Problem Decision Tree

Choosing the Right Algorithm



This decision tree covers the six canonical grid algorithm patterns. When in doubt, start by asking: "Do I need the shortest path, or just reachability?" — that single question determines BFS vs. DFS.

Use Case 1: Robot Maze Navigation

Autonomous Vehicles / Robotics

Context

A warehouse robot must navigate from charging dock to picking location. Warehouse floor as 2D grid; shelves = walls (1); open floor = passable (0).

Algorithm

BFS for shortest step count, or Dijkstra if different floor materials have traversal costs.

✓ **Production Detail:** Amazon Robotics (Kiva Systems) uses exactly this for their warehouse robots — the warehouse floor IS the grid graph.

Grid: (0=open, 1=shelf, S=start, E=end)

S 0 0 1 0

1 1 0 1 0

0 0 0 0 E

0 1 1 0 0

BFS gives:

- Shortest path length
- The actual path sequence

A* (Dijkstra + heuristic):

$f(n) = g(n) + h(n)$

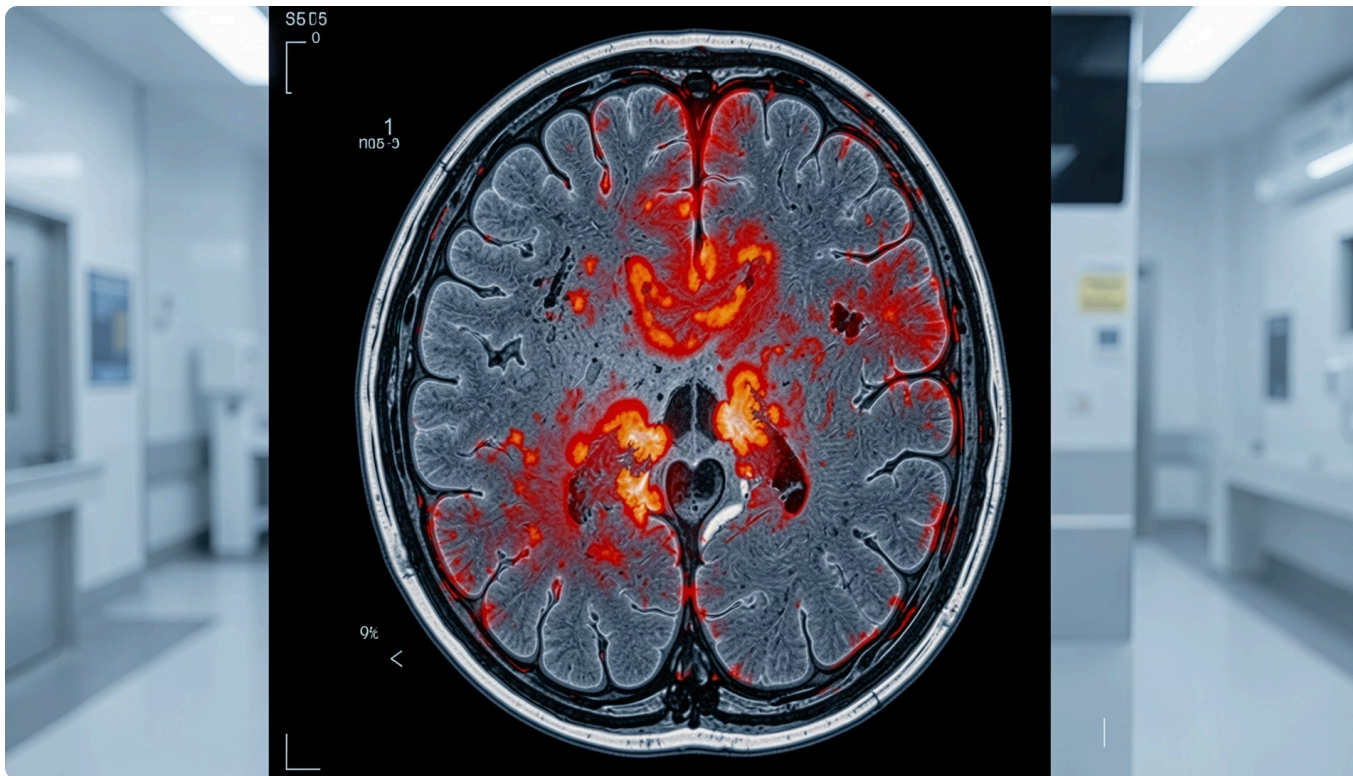
$h(n)$ = Manhattan distance to E

→ Faster search in large grids

→ Used in production for real-time navigation

Use Case 2: Image Segmentation

Computer Vision & Medical Imaging



Context

Image pixels as vertices; adjacent same-intensity pixels as edges. BFS/DFS for connected component labeling — assign each pixel a component ID.

Binary Image:

```
0 0 1 1 0
0 1 1 0 0
0 0 0 0 1
0 0 0 1 1
```

Connected 1-components:

Component 1: {(0,2),(0,3),(1,1),(1,2)}

Component 2: {(2,4),(3,3),(3,4)}

Medical Use: Each component = a candidate region of interest. Tumor detection in MRI relies on connected component labeling of abnormal-intensity pixel regions.

Use Case 3: Wildfire / Virus Spread Simulation

Emergency Management & Epidemiology



Context

Geographic regions as cells; adjacency = geographic neighbors. Multi-source BFS from initial fire/infection cells; BFS level = time step.

Grid: 0=safe, 1=burning, T=tree (fuel)

T T T T T

T 1 T T T ← Initial fire at (1,1)

T T T T T

T T T T T

After BFS step 1:

Fire spreads to all 4-adjacent trees

After BFS step 2:

Spreads further outward...

BFS level = minutes elapsed

→ Predict which regions affected and when

→ Optimize firefighting resource deployment

Use Case 4: Game Pathfinding — A* on Grid

Video Games: RTS, RPG, Tower Defense

Context

Game map tiles as vertices; movement directions as edges; movement costs as weights. A* (Dijkstra + heuristic) for fast shortest path in large game worlds.

Game **grid**:

0=walkable, 1=wall

2=water (cost 2), 3=mountain (cost 3)

A* with Manhattan distance heuristic:

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost to reach n

$h(n)$ = Manhattan distance to E
(admissible heuristic for grid)

A* explores nodes in order of $f(n)$

→ Finds optimal path faster than BFS

→ Prioritizes cells geometrically closer to the goal



Implementation 1: Number of Islands

Section 8: Production-Quality BFS Solution

```
FUNCTION num_islands(grid):
  IF NOT grid OR NOT grid[0]: RETURN 0

  ROWS = len(grid)
  COLS = len(grid[0])
  count = 0
  DIRS = [(-1,0), (1,0), (0,-1), (0,1)]

  FUNCTION bfs(start_r, start_c):
    queue = deque([(start_r, start_c)])
    grid[start_r][start_c] = '0' // Mark visited in-place
    WHILE queue not empty:
      (r, c) = queue.popleft()
      FOR (dr, dc) in DIRS:
        nr = r + dr; nc = c + dc
        IF 0 <= nr < ROWS AND 0 <= nc < COLS AND grid[nr][nc] == '1':
          grid[nr][nc] = '0' // Mark BEFORE enqueue
          queue.append((nr, nc))

  FOR r = 0 TO ROWS-1:
    FOR c = 0 TO COLS-1:
      IF grid[r][c] == '1':
        count += 1
        bfs(r, c)

  RETURN count
```

❌ **CRITICAL:** Mark visited WHEN ENQUEUEING, not when dequeuing. If you mark on dequeue, the same cell can be enqueued multiple times → $O(n^2)$ worst case instead of $O(n)$.

Implementation 2: Walls and Gates

Multi-Source BFS — Fill Each Room with Distance to Nearest Gate

```
// Grid: -1=wall, 0=gate, INF=empty room
// Fill each empty room with distance to nearest gate

FUNCTION walls_and_gates(grid):
    ROWS = len(grid); COLS = len(grid[0])
    INF = 2^31 - 1
    DIRS = [(-1,0),(1,0),(0,-1),(0,1)]
    queue = deque()

    // Initialize with ALL gates
    FOR r = 0 TO ROWS-1:
        FOR c = 0 TO COLS-1:
            IF grid[r][c] == 0:
                queue.append((r, c)) // All gates at distance 0

    // BFS from all gates simultaneously
    WHILE queue not empty:
        (r, c) = queue.popleft()
        FOR (dr, dc) in DIRS:
            nr = r + dr; nc = c + dc
            IF 0 <= nr < ROWS AND 0 <= nc < COLS AND grid[nr][nc] == INF:
                grid[nr][nc] = grid[r][c] + 1 // Distance propagates
                queue.append((nr, nc))

    // After: grid[r][c] = min distance to any gate (or INF if unreachable)
    // Time: O(ROWS × COLS) — each cell processed at most once
```

Implementation 3: Shortest Path in Binary Matrix

8-Directional BFS — LeetCode #1091

```
// Find shortest path from (0,0) to (n-1,n-1) through 0-cells (8-directional)
// Returns -1 if no path
```

```
FUNCTION shortest_path_binary_matrix(grid):
```

```
    n = len(grid)
```

```
    IF grid[0][0] == 1 OR grid[n-1][n-1] == 1: RETURN -1
```

```
    IF n == 1: RETURN 1
```

```
    DIRS8 = [(-1,-1),(-1,0),(-1,1),(0,-1),(0,1),(1,-1),(1,0),(1,1)]
```

```
    queue = deque([(0, 0, 1)])    // (row, col, path_length)
```

```
    grid[0][0] = 1                // Mark visited in-place
```

```
    WHILE queue not empty:
```

```
        (r, c, dist) = queue.popleft()
```

```
        FOR (dr, dc) in DIRS8:
```

```
            nr = r + dr; nc = c + dc
```

```
            IF 0 <= nr < n AND 0 <= nc < n AND grid[nr][nc] == 0:
```

```
                IF nr == n-1 AND nc == n-1: RETURN dist + 1
```

```
                grid[nr][nc] = 1 // Mark visited before enqueueing
```

```
                queue.append((nr, nc, dist + 1))
```

```
    RETURN -1
```

```
// Note: marking before enqueue is CRUCIAL for  $O(n^2)$  not  $O(n^4)$ 
```

Implementation 4: Pacific Atlantic Water Flow

Reverse Multi-Source BFS — LeetCode #417

```
// Find all cells that can flow to BOTH Pacific (top/left) AND Atlantic (bottom/right).  
// Key insight: Reverse BFS from ocean borders — water "climbs uphill" from oceans.
```

```
FUNCTION pacific_atlantic(heights):  
    ROWS = len(heights); COLS = len(heights[0])  
    DIRS = [(-1,0),(1,0),(0,-1),(0,1)]  
  
    FUNCTION bfs(starts):  
        visited = set(starts)  
        queue = deque(starts)  
        WHILE queue not empty:  
            (r, c) = queue.popleft()  
            FOR (dr, dc) in DIRS:  
                nr = r + dr; nc = c + dc  
                IF (nr,nc) not in visited AND valid(nr,nc):  
                    IF heights[nr][nc] >= heights[r][c]: // Uphill reverse  
                        visited.add((nr,nc))  
                        queue.append((nr,nc))  
        RETURN visited  
  
    pacific_starts = [(r,0) for r in range(ROWS)]  
                    + [(0,c) for c in range(COLS)]  
    atlantic_starts = [(r, COLS-1) for r in range(ROWS)]  
                     + [(ROWS-1, c) for c in range(COLS)]  
  
    pacific_reachable = bfs(pacific_starts)  
    atlantic_reachable = bfs(atlantic_starts)  
  
    RETURN pacific_reachable INTERSECTION atlantic_reachable
```

 **Reverse Thinking:** Instead of simulating water flowing downhill from each cell ($O(N^2)$), reverse the direction — BFS uphill from each ocean border ($O(N)$). Intersection = cells reachable by both.

Lab 1: Island Variants

Section 9: Progressive Difficulty — Build Intuition

Base Problem: Count Islands with BFS

Grid A:	Grid B:	Grid C:
1 1 0 0 0	1 1 1 1 0	1 1 0
1 1 0 0 0	1 1 0 1 0	0 1 0
0 0 1 0 0	1 1 0 0 0	0 1 1
0 0 0 1 1	0 0 0 0 0	

Expected: 3 Expected: 1 Expected: 1

Variant 2 — Island Perimeter

```
// Perimeter contribution per cell:  
// 4 (sides) - 2 × (number of land neighbors)  
FUNCTION island_perimeter(grid):  
    perimeter = 0  
    FOR each land cell (r, c):  
        perimeter += 4  
        FOR each land neighbor:  
            perimeter -= 2 // Shared edge loses 2 sides  
    RETURN perimeter
```

Variants

01

Max Island Area

Instead of counting islands, find the largest island (most cells). Track max component size during DFS/BFS.

02

Island Perimeter

For a grid with exactly one island, compute its perimeter. Each land cell contributes 4 minus 2× land neighbors.

03

Distinct Islands (Shape-Based)

Two islands are the same if one can be translated to match the other. Use DFS and record the relative path signature.

Lab 2: Rotting Oranges — Multi-Source BFS

Advanced: Spread Simulation with Edge Cases

Test Cases

Grid: 0=empty, 1=fresh, 2=rotten

Case 1:

Case 2:

Case 3:

2 1 1

2 1 1

0 2

1 1 0

0 1 1

0 1 1

1 0 1

Expected: 4

Expected: -1

Expected: 0

(all rot in 4 min)

(isolated fresh (no fresh oranges)

→ impossible)

Implementation Steps

01

Initialize Multi-Source Queue

Add ALL rotten oranges to queue at time 0 simultaneously

02

BFS Level-by-Level

Each BFS level = 1 minute elapsed; track max time

03

Scan for Remaining Fresh

After BFS: scan grid for any remaining 1 cells → return -1 if found

04

Return Total Minutes

Return elapsed time, or 0 if no fresh oranges existed initially



Edge Case: Multiple isolated rotten clusters all start simultaneously — they're all in the initial queue. Don't process them sequentially.

Performance Considerations

Section 11: Grid Size vs. Algorithm Complexity

Algorithm	Time	Space
BFS / DFS on grid	$O(N)$	$O(N)$
Multi-source BFS	$O(N)$	$O(N)$
Dijkstra on grid	$O(N \log N)$	$O(N)$
0-1 BFS on grid	$O(N)$	$O(N)$
Floyd-Warshall on grid	$O(N^2)$ ❌	$O(N^2)$ ❌

⊗ **For a 1000×1000 grid ($N=10^6$):** BFS/DFS = 10^6 ops (fast). Floyd-Warshall = 10^{12} ops (completely impractical). Never use all-pairs shortest path on grids.

The "Mark Before Enqueue" Rule — Complexity Impact

```
// WRONG — mark on dequeue:
queue.append((r, c))
// ... later ...
(r, c) = queue.popleft()
visited[r][c] = TRUE // Too late!
// Same cell already enqueued multiple times.

// CORRECT — mark on enqueue:
visited[r][c] = TRUE // Prevent re-enqueue
queue.append((r, c))

// Impact on complexity:
// Mark on dequeue:  $O(N \times \text{branching factor})$ 
// → cells enqueued multiple times
// Mark on enqueue:  $O(N)$ 
// → each cell enqueued exactly once
// In 4-directional grid, marking on dequeue
// can be 4× slower.
```

Best Practices — Matrix as Graph

Section 15: Do's and Don'ts

✓ DO'S

Category	Practice
Design	Always use a direction array (DIRS) — never hard-code 4 neighbor checks
Correctness	Bounds check EVERY neighbor: $0 \leq nr < \text{ROWS}$ AND $0 \leq nc < \text{COLS}$
Performance	Mark cells as visited WHEN ENQUEUEING, not when dequeuing
Design	For "nearest source" problems, use multi-source BFS — initialize all sources in queue
Implementation	Prefer BFS over DFS for shortest path on grids
Robustness	Handle edge cases: 0×0 grid, 1×1 grid, all walls, no walls
Production	For large grids ($>10^5$ cells), use iterative BFS — recursive DFS risks stack overflow

✗ DON'TS

Category	Anti-Pattern
Correctness	Don't skip bounds check — #1 cause of crashes in grid problems
Performance	Don't use DFS for shortest-path problems — BFS guarantees minimum steps
Design	Don't build an explicit adjacency list for grid problems — compute neighbors on-the-fly
Correctness	Don't forget to handle disconnected grids — loop over ALL cells, not just from (0,0)
Performance	Don't use Floyd-Warshall on grids — $O(N^2)$ or $O(N^3)$ kills performance

Common Mistakes — Matrix as Graph

Section 16: Pitfalls by Level

Level	Mistake	Why	Impact	Fix
Intermediate	Missing bounds check	Confidence after first few problems	Index out of bounds crash	Always check <code>0 <= nr < ROWS</code> AND <code>0 <= nc < COLS</code> first
Intermediate	Marking visited on dequeue	Natural intuition	TLE — cells enqueued multiple times	Mark visited immediately when adding to queue
Advanced	Using DFS for shortest path	"DFS is simpler"	Returns non-minimal path	BFS for unweighted shortest path; Dijkstra for weighted
Advanced	Single-source BFS for "nearest gate"	Not recognizing multi-source pattern	$O(N \times \text{Gates})$ instead of $O(N)$	Initialize ALL sources in queue at distance 0
Advanced	Forgetting to check for remaining fresh oranges after BFS	BFS terminates normally even if some unreachable	Returns wrong answer (time instead of -1)	After BFS, scan grid for any remaining 1 cells → return -1
Advanced	Using <code>visited</code> set with tuple (r,c) for large grids	Correct but slow	Hash overhead for large grids	Use 2D boolean array <code>visited[r][c]</code> instead

Matrix as Graph — Learning Summary

Section 18: Key Takeaways, Pattern Reference & Next Steps

Implicit Graph

A 2D matrix is an implicit graph — cells are vertices, adjacency derived on-the-fly using a direction array. Never build an explicit adjacency list.

Direction Array

`DIRS = [(-1,0),(1,0),(0,-1),(0,1)]` is the universal building block. Always bounds-check before accessing neighbors.

Mark on Enqueue

Critical for $O(N)$ complexity. Marking on dequeue allows duplicate enqueues $\rightarrow O(N \times \text{branching})$ worst case.

Multi-Source BFS

Initialize ALL sources in queue at distance 0 for "nearest source" problems. BFS level = time step for spread simulations.

Recommended Next Topics

→ BFS Deep Dive

Level-order BFS, bidirectional BFS, BFS on implicit graphs

→ Dijkstra's Algorithm

Single-source shortest path on weighted graphs and grids

→ Union-Find / DSU

Alternative to BFS/DFS for component detection; efficient for dynamic graphs

→ A* Search

Heuristic-guided BFS for game pathfinding and large-grid optimization

→ Graph DP

DP on DAGs, longest path, counting paths on grids